

BÀI TẬP VỀ NHÀ NESTJS – SỐ 01

LỜI GIẢI CHI TIẾT PHẦN A & B

PHẦN A — Lời giải Lý thuyết

1. @Injectable() cho phép Nest "tự tạo và tự truyền" một instance mà bạn không hề gọi new. Cơ chế nào đứng sau chuyện đó?

► Cơ chế đứng sau là Dependency Injection (DI) Container (hoặc Inversion of Control - IoC Container). Khi ứng dụng khởi động, NestJS quét các class có @Injectable(), tự động gọi lệnh 'new' để tạo ra các instance (bản sao) và lưu giữ chúng trong một bộ nhớ trung tâm (Container). Khi một Controller hay Service khác cần dùng, Container sẽ tự động "truyền" (inject) instance đó vào constructor mà dev không cần tạo tay.

2. @Module() có providers lẫn exports. Một provider nằm trong providers nhưng không nằm trong exports thì module khác import được xảy ra chuyện gì? Vì sao thiết kế "mặc định đóng, phải khai báo mới mở" lại an toàn hơn?

► Nếu không có trong 'exports', module khác sẽ không thể dùng được provider đó (Nest sẽ ném lỗi can't resolve dependencies).

► Thiết kế "mặc định đóng" (Encapsulation) giúp kiểm soát chặt chẽ phạm vi truy cập (scope) của code, tránh rò rỉ dữ liệu, giảm thiểu các module bị phụ thuộc chéo lộn xộn (spaghetti code) và giúp dễ dàng debug hơn khi dự án lớn lên.

3. Vì sao logic nghiệp vụ (business logic) không nên nằm trong Controller? Nêu hai hậu quả cụ thể nếu bạn viết thẳng code truy vấn database trong hàm xử lý route.

► Controller chỉ nên làm nhiệm vụ "Tiếp tân": nhận request HTTP và trả về response. Logic nghiệp vụ phải đưa vào Service.

► Hậu quả 1: Khó viết Unit Test. Bạn không thể test logic tính toán mà không dính tới việc giả lập các object Request/Response (Req/Res) cồng kềnh của Express.

► Hậu quả 2: Không thể tái sử dụng code. Nếu mai này bạn muốn tái sử dụng đoạn logic truy vấn đó cho một cron-job chạy ngầm hoặc qua giao thức WebSockets thay vì HTTP, bạn sẽ phải copy-paste lại toàn bộ.

4. Tạo tài nguyên thành công thường trả 201, không phải 200. Sự khác biệt đó nói lên điều gì với client? Khi nào một endpoint POST vẫn nên trả 200 thay vì 201?

► HTTP 201 (Created) báo hiệu rõ ràng cho client rằng một tài nguyên MỚI đã được tạo ra thành công trên server (thường đi kèm dữ liệu id mới trả về).

► POST trả về HTTP 200 (OK) khi request đó dùng để thực thi một hành động xử lý/tính toán (như login, tính toán thống kê) mà không thực sự tạo ra thêm bản ghi mới nào trong Database.

5. GET /drinks/:id với id không tồn tại và id sai định dạng (ví dụ "abc") - vì sao cả hai nên trả cùng 404, thay vì cái sau trả 400?

► Dù là id không có thực hay id sai định dạng (không phải ObjectId của Mongo), bản chất đối với client bên ngoài đều là "không tìm thấy món nước đó". Việc trả 400 (Bad Request) do sai định dạng ObjectId là làm lộ chi tiết kỹ thuật (implementation detail) của Database ra ngoài API. Tầng API chỉ nên giao tiếp bằng business logic: "Không tìm thấy -> 404".

6. Route param (:id) và query param (?keyword=) đều mang dữ liệu từ URL. Nên quy tắc chọn cái nào.

- ▶ Route param (:id): Dùng khi muốn xác định (định danh) chính xác MỘT tài nguyên cụ thể bắt buộc phải có. VD: Lấy thông tin user số 5 -> /users/5.
- ▶ Query param (?keyword=): Dùng khi muốn cung cấp các tham số tùy chọn để LỌC, PHÂN TRANG, hoặc SẮP XẾP trên một TẬP HỢP tài nguyên. VD: Tìm user tên A -> /users?name=A.

7. Cơ sở dữ liệu quan hệ tổ chức dữ liệu thành bảng - hàng - cột, có khóa ngoại nối bảng này với bảng khác. MongoDB tổ chức thành collection – document. Nên một tình huống dữ liệu (ví dụ: Recipe có mảng ingredients) mà mô hình document xử lý gọn hơn mô hình quan hệ, và giải thích vì sao.

- ▶ Tình huống: Món trà sữa (Drink) có nhiều Topping đi kèm. Ở CSDL quan hệ (SQL), bạn phải tạo 2 bảng Drink và Topping, nối bằng khóa ngoại (Foreign Key) và dùng lệnh JOIN phức tạp để query.
- ▶ Với Document (MongoDB): Topping được nhúng (embed) thẳng thành một mảng object bên trong document Drink. Khi lấy data, chỉ cần 1 lần đọc disk là lấy được toàn bộ thông tin món và topping cực kỳ nhanh gọn.

8. @Schema()/@Prop() của Mongoose trông giống hệt việc "ép schema" — nhưng MongoDB tự thân không ép schema ở tầng lưu trữ. Vậy Mongoose Schema đang bảo vệ ở tầng nào? Nên một lợi ích và một thứ bạn không mất đi so với dùng MongoDB thuần.

- ▶ Bảo vệ ở Tầng Ứng dụng (Application Layer - Mongoose/NestJS), không phải tầng Database.
- ▶ Lợi ích: Phát hiện và chặn dữ liệu rác (thiếu field bắt buộc, sai kiểu dữ liệu) ngay từ trong code trước khi gửi xuống DB; hỗ trợ tự động sinh Type gợi ý code cho TypeScript.
- ▶ Không mất đi: Sự linh hoạt (Flexible Schema) của NoSQL. Bạn vẫn có thể dễ dàng khai báo thêm trường mới vào code mà không cần phải viết file migration sửa cấu trúc bảng phức tạp như SQL.

9. Vì sao ứng dụng không nên trả thẳng field _id (kiểu ObjectId) ra API cho client, mà cần "transform" thành id dạng chuỗi? Liên hệ với lý do DTO tồn tại.

- ▶ Việc trả thẳng _id (dấu gạch dưới) và kiểu ObjectId (Object lồng nhau) ra ngoài làm phá vỡ chuẩn RESTful API, đồng thời làm rò rỉ việc hệ thống đang xài MongoDB cho Frontend biết.
- ▶ Liên hệ DTO (Data Transfer Object): DTO và toJSON transform đóng vai trò là "lớp màng lọc biên giới". Chúng giúp giấu đi cấu trúc lộn xộn bên dưới Database, chỉ trả ra cho Frontend những trường dữ liệu sạch sẽ, đúng chuẩn (id kiểu string) mà Frontend cần.

10. findById("khong-hop-le") ném CastError thay vì trả null. Nên hậu quả nếu không bắt lỗi này, và cách phòng tránh mà không cần thư viện validate ngoài.

- ▶ Hậu quả: App sẽ bị crash hoặc NestJS sẽ tự động hứng lỗi và ném ra mã HTTP 500 (Internal Server Error) cho client. Điều này là sai vì lỗi không phải do sập server, mà do client gửi id bậy (phải là 404).
- ▶ Cách phòng tránh: Trước khi chạy lệnh query, dùng lệnh kiểm tra Types.ObjectId.isValid(id) của chính thư viện mongoose. Nếu false thì trả về null ngay lập tức.

11. Connection string MongoDB nằm trong .env, không hardcode trong code. Nên hai lý do (1 bảo mật, 1 không bảo mật).

- ▶ Lý do bảo mật: Đẩy code lên GitHub bị lộ password, hacker sẽ chiếm quyền truy cập toàn bộ Database và đánh cắp dữ liệu.
- ▶ Lý do không bảo mật: Giúp ứng dụng dễ dàng triển khai (deploy) ở nhiều môi trường khác nhau. Trên

máy Dev dùng DB local, đưa lên máy thật (Production) dùng DB Atlas mà không cần phải sửa và build lại source code.

12. onModuleInit() chỉ seed dữ liệu mẫu khi countDocuments() === 0. Mô tả chuyện gì xảy ra nếu bỏ điều kiện đó và seed vô điều kiện mỗi lần app khởi động lại.

► Mỗi khi bạn gõ code và lưu lại (Ctrl+S), môi trường phát triển (npm run start:dev) sẽ tự động khởi động lại server. Nếu không có điều kiện check số lượng, mỗi lần restart nó sẽ lại chèn thêm 3 món mới vào Database. Sau 10 lần lưu file, DB của bạn sẽ chứa 30 bản ghi trùng lặp (rác data).

13. Định nghĩa "seam" (đường nối) trong kiến trúc phần mềm bằng lời của riêng bạn. Vì sao DrinkService gọi thẳng Model của Mongoose là không có seam, còn gọi qua interface DrinkRepository là có seam?

- Seam (Đường nối): Là một điểm nối lỏng lẻo trong code, cho phép bạn tháo lắp/thay thế một linh kiện bên dưới mà không làm ảnh hưởng đến linh kiện bên trên.
- Gọi thẳng Model Mongoose là KHÔNG có seam vì Service đang bị trói chặt (tight-coupled) vào các hàm đặc thù của thư viện mongoose (như .findById().exec()).
- Gọi qua interface DrinkRepository CÓ seam vì Service chỉ biết gọi hàm chung chung (như .findOne()). Bên dưới dùng Mongoose hay PostgreSQL thì Service hoàn toàn không cần quan tâm.

14. Sau khi compile TypeScript sang JavaScript, interface biến mất hoàn toàn. Vậy NestJS dựa vào đâu để biết "ai cần cái gì" lúc chạy? Injection token giải quyết vấn đề đó thế nào?

- JS không có interface. Để giải quyết, Nest dùng "Injection Token" (thường là một Symbol hoặc chuỗi String) làm biến số định danh.
- Cách giải quyết: Ta đăng ký với Nest một cái thẻ tên (Token = DRINK_REPOSITORY) gắn với class MongooseDrinkRepository. Khi Service khởi tạo, nó sẽ hô lên "Nest ơi, đưa cho tôi thằng nào đang cầm cái Token này (@Inject(Token))", Nest sẽ tự hiểu và bơm đúng class Mongoose vào.

15. Giả sử mai bạn đổi từ MongoDB sang PostgreSQL. Với cách viết "Service gọi thẳng Model", bạn phải sửa bao nhiêu file? Với cách viết "qua interface + token", câu trả lời khác thế nào?

- "Service gọi thẳng Model": Bạn sẽ phải sửa ban hành toàn bộ file Service (Tầng nghiệp vụ), thay đổi mọi lệnh query mongoose sang lệnh SQL.
- "Qua interface + token": File Service KHÔNG CẦN SỬA MỘT CHỮ NÀO. Bạn chỉ việc tạo 1 file repository mới (PostgresDrinkRepository implements DrinkRepository) để viết lệnh SQL, sau đó vào file Module đổi nhẹ chữ 'useClass' trở sang class mới là xong. Tầng nghiệp vụ hoàn toàn được bảo vệ.

PHẦN B — Tìm lỗi trong code

Dưới đây là lời giải chi tiết cho 10 đoạn mã bị lỗi trong bài.

Lỗi số 1

```
@Injectable()
export class DrinkService {
  constructor(private readonly drinkModel: Model<DrinkDocument>) {}

  findAll(): Promise<Drink[]> {
    return this.drinkModel.find().exec();
  }
}
```

```
}
```

- Lỗi ở đâu: Thiếu decorator `@InjectModel()` trong constructor.
- Tại sao: Model của Mongoose không phải là một Provider thông thường của NestJS. Bạn phải dùng `@InjectModel(Drink.name)` để ra lệnh cho NestJS bơm đúng Model đã được biên dịch từ schema vào biến `drinkModel`.
- Sửa lại:
`constructor(@InjectModel(Drink.name) private readonly drinkModel: Model<DrinkDocument>) {}`

Lỗi số 2

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.drinkService.findOne(+id);
}
```

- Lỗi ở đâu: Dùng dấu `+id` để ép kiểu tham số sang dạng số Number.
- Tại sao: ID của MongoDB là chuỗi ObjectId (vd: '64b1f...'). Việc ép kiểu nó sang Number (`+id`) sẽ biến nó thành NaN (Not a Number), làm lệnh query bị hỏng hoàn toàn.
- Sửa lại:
`return this.drinkService.findOne(id);`

Lỗi số 3

```
async findOne(id: string): Promise<Drink> {
  const drink = await this.drinkModel.findById(id).exec();
  if (!drink) {
    throw new NotFoundException(`Drink with id ${id} not found`);
  }
  return drink;
}
```

- Lỗi ở đâu: Gọi `findById(id)` ngay lập tức mà không kiểm tra định dạng id.
- Tại sao: Nếu id truyền vào không phải là chuỗi ObjectId hợp lệ, mongoose sẽ văng lỗi `CastError` ngay tại dòng `await`, làm sập request và trả về lỗi 500 thay vì xuống được dòng ném lỗi 404 (NotFound).
- Sửa lại:
`const drink = Types.ObjectId.isValid(id) ? await this.drinkModel.findById(id).exec() : null;`

Lỗi số 4

```
async onModuleInit(): Promise<void> {
  await this.drinkModel.insertMany(SEED_DRINKS);
}
```

- Lỗi ở đâu: Chèn dữ liệu (seed) vô điều kiện.
- Tại sao: Cứ mỗi lần server khởi động (đặc biệt lúc dev lưu file), nó lại chèn thêm data mẫu vào làm DB bị rác. Phải đếm xem DB đã có data chưa trước khi chèn.
- Sửa lại:

```
const count = await this.drinkModel.countDocuments();
if (count === 0) await this.drinkModel.insertMany(SEED_DRINKS);
```

Lỗi số 5

```
@Schema()
export class Drink {
  @Prop({ required: true })
  name!: string;

  @Prop({ type: [ToppingSchema], default: [] })
  toppings!: Topping[];
}
```

- Lỗi ở đâu: Định nghĩa mảng ToppingSchema bị sai cú pháp type.
- Tại sao: Mongoose và NestJS yêu cầu khai báo schema lồng nhau (Subdocument) trong mảng phải đưa vào thuộc tính type: [ToppingSchema]. Khai báo như đề bài là sai chuẩn.
- Sửa lại:
@Prop({ type: [ToppingSchema], default: [] })
toppings!: Topping[];
// LƯU Ý: Đoạn code gốc trong đề thực chất KHÔNG CÓ LỖI SYNTAX ở Mongoose mới, nhưng lỗi phổ biến hay gặp là quên truyền type cho mảng subdocument.

Lỗi số 6

```
@Schema()
export class Topping {
  @Prop({ required: true })
  name!: string;
  @Prop({ required: true })
  quantity!: number;
}
export const ToppingSchema = SchemaFactory.createClass(Topping);
```

- Lỗi ở đâu: Lớp Topping (Subdocument) thiếu tùy chọn {_id: false}.
- Tại sao: Mặc định Mongoose sẽ tự động sinh thêm 1 trường _id cho MỌI phần tử nằm trong mảng toppings. Điều này là dư thừa và làm rác data vì topping không cần truy vấn độc lập.
- Sửa lại:
@Schema({ _id: false })
export class Topping {

Lỗi số 7

```
@Module({
  imports: [
    MongooseModule.forRootAsync({
      useFactory: () => ({ uri: process.env.MONGODB_URI }),
    }),
  ],
})
```

```
export class AppModule {}
```

- Lỗi ở đâu: Đọc biến môi trường process.env trực tiếp trong cấu hình Mongoose.
- Tại sao: Ở ứng dụng NestJS chuẩn, việc đọc env phải thông qua ConfigModule và ConfigService để đảm bảo thứ tự nạp biến môi trường an toàn. Đọc trực tiếp có thể dẫn đến việc biến bị undefined lúc server vừa start.
- Sửa lại:
imports: [ConfigModule.forRoot(), MongooseModule.forRootAsync({ inject: [ConfigService], useFactory: (cfg: ConfigService) => ({ uri: cfg.get('MONGODB_URI') }) })]

Lỗi số 8

```
@Delete('/:id')  
@HttpCode(204)  
remove(@Param('id') id: string) {  
  this.drinkService.remove(id);  
}
```

- Lỗi ở đâu: Quên từ khoá await và async trước lệnh gọi hàm service xoá database.
- Tại sao: Hàm remove() chọn xuống database trả về Promise. Nếu không có await, Controller sẽ trả ngay HTTP 204 về cho client trong khi lệnh xoá DB chưa kịp chạy xong. Gây ra các lỗi bất đồng bộ rất khó lường.
- Sửa lại:
async remove(@Param('id') id: string) {
 await this.drinkService.remove(id);
}

Lỗi số 9

```
@Module({  
  providers: [{ provide: DrinkService, useClass: MongooseDrinkRepository }],  
  controllers: [DrinkController],  
})  
export class DrinkModule {}
```

- Lỗi ở đâu: Định nghĩa sai Provider Token (provide: DrinkService).
- Tại sao: Nếu khai provide là DrinkService, thì khi controller gọi DrinkService, Nest sẽ bơm thẳng MongooseDrinkRepository vào. Điều này phá vỡ hoàn toàn cấu trúc Repository Pattern. Token khai báo phải là một interface/symbol độc lập.
- Sửa lại:
providers: [DrinkService, { provide: DRINK_REPOSITORY, useClass: MongooseDrinkRepository }]

Lỗi số 10

```
@Injectable()  
export class DrinkService {  
  constructor(  
    @Inject(DRINK_REPOSITORY) private readonly repo: MongooseDrinkRepository,  
  ) {}  
}
```

```
}
```

- ▶ Lỗi ở đâu: Định nghĩa kiểu dữ liệu cho biến repo là MongooseDrinkRepository.
- ▶ Tại sao: Nếu định nghĩa cứng class Mongoose ở đây, tầng Service sẽ lại bị trói chặt vào Mongoose, làm mất hoàn toàn tác dụng của "đường nối - Seam". Tầng Service chỉ được phép biết đến Interface.
- ▶ Sửa lại:

```
constructor(@Inject(DRINK_REPOSITORY) private readonly repo: DrinkRepository) {}
```